E12 — Number of Islands (BFS & DFS)

Experiment: 12 | **LeetCode:** #200 | **CO:** CO2, CO3 | **BT Level:** BT5

Problem Statement

Given an `m x n` 2D binary grid `grid` which represents a map of `'1'`s (land) and `'0'`s (water), return the number of islands.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are surrounded by water.

Example 1:

```
Input:
grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input:
grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- `m == grid.length`, `n == grid[i].length`
- `1 ≤ m, n ≤ 300`
- `grid[i][j]` is `'0'` or `'1'`

Concept: Graph View of the Grid

Model the grid as an **implicit graph:**

- Each cell `(r, c)` with value `'1'` is a **node**
- Two land cells are **connected** if they are horizontally or vertically adjacent
- An **island** = a **connected component** of land cells

Strategy: Iterate over every cell. When a `'1'` is found, increment the island count and flood-

fill (BFS or DFS) to mark all connected land cells as visited (so they are not counted again).

Solution 1: DFS (Recursive Flood Fill)

```
def numIslands_dfs(grid):
    """
    DFS flood-fill — marks visited cells by overwriting '1' with '0'.
    Time: O(m * n)  |  Space: O(m * n) for recursion stack
    """
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        # Base case: out of bounds or water/visited
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1
            return
        grid[r][c] = '0'   # Mark visited (sink the land)
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)   # Flood-fill the entire island

    return count
```

Solution 2: BFS (Queue-based Flood Fill)

```
from collections import deque

def numIslands_bfs(grid):
    """
    BFS flood-fill using a queue.
    Time: O(m * n)  |  Space: O(min(m, n)) for queue in worst case
    """
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for r in range(rows):
```

```
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                grid[r][c] = '0'   # Mark as visited immediately
                queue = deque([(r, c)])

                while queue:
                    row, col = queue.popleft()
                    for dr, dc in directions:
                        nr, nc = row + dr, col + dc
                        if 0 <= nr < rows and 0 <= nc < cols and grid[n
                            grid[nr][nc] = '0'   # Mark visited before
                            queue.append((nr, nc))

    return count
```

Solution 3: Union-Find (Disjoint Set)

```
class UnionFind:
    def __init__(self, grid):
        rows, cols = len(grid), len(grid[0])
        self.parent = {}
        self.count = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    self.parent[(r, c)] = (r, c)
                    self.count += 1

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])   # Path compres
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx != ry:
            self.parent[rx] = ry
            self.count -= 1

def numIslands_uf(grid):
    """
    Union-Find approach.
    Time: O(m * n * α(m*n)) ≈ O(m * n)  |  Space: O(m * n)
    """
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    uf = UnionFind(grid)
```

```
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                grid[r][c] = '0'
                for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][n
                        uf.union((r, c), (nr, nc))

    return uf.count
```

Detailed Trace

Example 2: 3 Islands

```
Grid (initial):
  1 1 0 0 0
  1 1 0 0 0
  0 0 1 0 0
  0 0 0 1 1
```

BFS Trace:

```
(0,0) = '1' → count=1, enqueue (0,0), mark '0'
  Process (0,0): neighbors (1,0)→'1', (0,1)→'1'  → enqueue both, mark '
  Process (1,0): neighbors (0,0)→'0', (2,0)→'0', (1,1)→'1' → enqueue (1
  Process (0,1): neighbors (0,0)→'0', (0,2)→'0', (1,1)→'0' → nothing
  Process (1,1): no unvisited land neighbors
Queue empty. Island 1 fully visited.

Grid now:
  0 0 0 0 0
  0 0 0 0 0
  0 0 1 0 0
  0 0 0 1 1

(2,2) = '1' → count=2, enqueue (2,2), mark '0'
  Process (2,2): no unvisited land neighbors
Queue empty. Island 2 fully visited.

(3,3) = '1' → count=3, enqueue (3,3), mark '0'
  Process (3,3): neighbor (3,4)→'1' → enqueue (3,4), mark '0'
  Process (3,4): no unvisited land neighbors
Queue empty. Island 3 fully visited.

return 3 ✓
```

Complexity Summary

Approach	Time	Space

DFS (recursive)	$O(m \times n)$	$O(m \times n)$ — recursion stack
BFS (queue)	$O(m \times n)$	$O(\min(m, n))$
Union-Find	$O(m \times n)$	$O(m \times n)$

Key Takeaways

- **DFS** is the simplest to implement; risk of stack overflow on very large grids.
- **BFS** avoids deep recursion; queue size at most proportional to the perimeter of an island.
- **Union-Find** is useful when the grid is dynamic (cells changing from water to land).
- Marking visited cells in-place (`'1' → '0'`) avoids extra memory for a visited set.